



Tutorial: Project Research and Development

Instructions

- This project is a **group project**.
- The project is 20% of the total mark for the course.
- The details of the marking scheme will be communicated later.
- The estimated workload is 30 hours per member in total, or 6 hours per week for 5 weeks, per project member.
- Submit your paper by **Sunday, 12 April 2026, 23:59** to EasyChair.
- Submit your video by **Sunday, 12 April 2026, 23:59** to Canvas.
- Submit your reviews (*individual submission*) by **Friday, 17 April 2026, 23:59** to EasyChair.

The University takes a strict view of plagiarism and considers it a serious form of academic dishonesty. Any student found to have engaged in such misconduct will be subjected to disciplinary action by the University. Please refer to the NUS Plagiarism Policy. The University prohibits cheating in any form during assessments, tests, quizzes or examinations. Such acts will be considered, at minimum, as “Moderate” offences, resulting in the default sanction of a ‘Fail’ grade for the entire course.

Topics

Each group chooses one topic from the list below and indicates its choice in the GoogleSheet indicated in Canvas. **At most, 3 groups can choose each topic on a first-come-first-served basis.**

1. **Entity-Relationship Graphical Interface.** Develop an interactive drawing tool facilitating the creation of *Entity-Relationship Diagram (ERD)* and the corresponding *SQL DDL* code generation. The tool supports various notations, including those from lectures (*i.e., multiple candidate keys, etc*), UML, and logical diagrams. It allows conversion between the different notations and support for multiple target DBMS and versions, accommodating system-specific syntax and additional features. Development options include a service, a JavaScript applet, or a standalone software (*e.g., in Python*), with potential Cloud service or virtual machine deployment.
2. **Provenance-Aware Relational Algebra.** Develop a provenance-aware relational algebra evaluator for a subset of relational algebra expressions (*selection, projection, cross product, multiset sum, deduplication, and potentially aggregation*). The semantics is *multiset* semantics. Provenance tracking is done using *provenance semiring framework* and includes Boolean function semiring to support probabilistic database. The project includes benchmark to test the correctness of provenance tracking as well as to evaluate the performance of the evaluator (*comparison with and without provenance*) alongside the implementation.
3. **Leaderboard.** Develop an online platform for SQL performance competitions, allowing users to submit queries and compete based on *correctness* and *efficiency*. The platform –implemented for PostgreSQL– features additional functionalities such as timeouts, multiple database instance testing, and query parsing. Deployment options include a Cloud service or virtual machine deployment.
4. **Large Language Model Optimization of SQL Query.** Illustrate and evaluate the capabilities of large language model (*LLM*) to rewrite complex SQL queries that suffers from slow executions despite query optimization. The research involves prompt engineering and the criteria that makes a query challenging for LLM to optimize. The evaluation (*positive/negative*) is done on standard benchmark such as TPC benchmark (*e.g., TPC-DS*) comparing the performance against other query rewriting tools (*e.g., pg_query*).
5. **Fake but Realistic Data.** Design, implement, and demonstrate a tool for generating realistic random data for entity-relationship designs. The tool considers participation constraints, join selectivity, and probability distributions. The report and presentation cover background concepts such as cardinality, participation, and selectivity in relational database.
6. **Machine Learning for Cost Estimator.** Design, train, and comparatively evaluate machine learning models to estimate the cost, planning time, and execution time of SQL queries with PostgreSQL. Investigate opportunities to design a machine learning model assisting the formulation of efficient queries.
7. **Text-Based Entity-Relationship Model.** Design a *declarative* language to describe *Entity-Relationship Diagram (ERD-Language)* and another *declarative* language to query the given ERD (*ERD-Query*). The ERD-language supports the ERD notation from the lecture and the ERD-Query supports common constraints query on identification, participation, and relationship.

8. **Why-Not Explanations of Query Answers in PostgreSQL.** Develop a system (*e.g.*, *query rewriting*, *relational algebra*, *etc*) to explain why some data is missing from the output (*i.e.*, *Why-Not provenance*). The project includes benchmark to test the correctness of Why-Not explanation as well as to evaluate the performance of the evaluator (*comparison with and without Why-Not analysis*) alongside the implementation.
9. **The Chase.** Create an interactive tool implementing the Chase algorithm for functional dependencies. The tool offers functionalities such as entailment, lossless decomposition, and minimal cover generation. The project includes examples, benchmarks, and may offer multiple Chase algorithm versions (*e.g.*, *including multi-valued dependencies*). The report provides theoretical background coverage. Deployment options include a Cloud service or virtual machine deployment.
10. **Large Language Model for Database Design and Programming.** Illustrate and evaluate the capabilities of large language model (*LLM*) to answer database design and programming questions. The questions may cover all or part of the syllabus of an introductory course on the design and implementation of database applications with relational database systems and SQL for computing students. The topic includes: entity-relationship modelling, SQL data definition, manipulation, and query languages, stored procedures and triggers, as well as normalization. The research involves prompt engineering and the identification of criteria that makes a query easy or challenging for an LLM. The evaluation involves testing the LLM capabilities with questions from at least one popular textbook (*e.g.*, *Database Management Systems by Ramakrishnan, Raghu, and Gehrke*).
11. **Relax and Find the Key.** Develop an online game centered around identifying candidate keys of relational schemas with functional dependencies. The game involves the random generation of problems of different difficulty levels (*to be defined theoretically*). Deployment options include a Cloud service or virtual machine deployment. The report and presentation present the theoretical background and the game.
12. **Monte-Carlo Sampling Normalization.** Investigate the distribution of minimal covers and normal forms of a relation with functional dependencies using Monte-Carlo sampling techniques. The project involves understanding and implementing algorithms for the uniform generation of sets of functional dependencies at random, the computation of minimal covers, and the testing of normal forms.
13. **Check Constraints Compiler.** Design and implement a PostgreSQL compiler that translates **CHECK** SQL constraints into *triggers* and *stored procedures*. The project includes performance evaluation alongside the implementation.

Paper

Your work and results are presented in a paper. The paper's submission is managed as a scientific and technical conference. The paper consists of six (6) sections, followed by a list of references.

1. The *Title* summarizes in a few words the main idea of the work.
2. The *Abstract* overviews in a few lines, the motivation and the work. It also announces the main result or results.
3. The *Introduction* section presents the challenge and outlines the work.
4. The *Background* section presents the domain and technical textbook knowledge necessary to understand the paper.
5. The *Related Work* section surveys the related work in order to justify the choice of the selected technique.
6. The *Methodology* section presents the selected technique.
7. The *Performance Evaluation* section presents the experimental set-up, including data sets and metrics, as well as presents and analyzes the results of the comparative empirical performance evaluation.
8. The *Conclusion* section summarizes the work and results.
9. The *References* lists the sources cited in the paper.

The paper is no longer than fifteen (15) pages, including figures and references, in Portable Document Format ([.pdf](#)). The paper is written in LaTeX¹ using Overleaf² (www.overleaf.com) and follows Springer Lecture Notes in Computer Science's template. The group leader submits the paper to the CS4221/CS5421 conference created for this purpose in EasyChair³ (easychair.org). You will receive an invitation to easychair.org and to the CS4221/CS5421 program committee in due course.

Video

The video summarizes the work and results as would the presentation of a paper in a scientific and technical conference.

1. The video consists of slides and comments.
2. The video follows the same structure as the report.
3. The video may include other relevant content such as demonstrations of the model.
4. The video must be in MP4 ([.mp4](#)) or in QuickTime ([.mov](#)) file format no longer than 10 minutes and no larger than 200MB.

You are not allowed to artificially speed the video but you may use handbrake.fr to reduce the file size, if needed.

¹LaTeX is a typesetting system widely used for scientific and technical writing.

²Overleaf is an online collaborative platform for LaTeX document creation. Overleaf has a user-friendly interface that simplifies LaTeX usage, enables real-time collaboration and facilitates access to various templates and tools.

³EasyChair is an online conference management system designed to streamline the submission and review process.

Reviews

The project evaluation is managed as a scientific and technical conference.

1. Each student is assigned to review 3 papers.
2. Download the papers assigned to you in CS4221/CS5421 conference in EasyChair.
3. Read and review the papers following the guidelines that will be communicated to you.
4. Submit your reviews to the CS4221/CS5421 conference in EasyChair.